

0.18.1. Avancer avec `deplace_x` — de base, ailleurs

Le 0.18 sur la ligne 8, avec le B.

1. Ce qu'il faut comprendre

C'est le **0.18**, avec une autre lettre, le **B** (`ALPHABET[1]`, la 2e lettre), et une autre place : la **ligne 8** au lieu de 0 (2e réglage de `deplace_x`).

C'est la **version de base** : une seule lettre, à la deuxième place. Le 0.18.2 met les deux ensemble.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_x>` → [0.17.3. La fonction `deplace\_x\(\)` — avancer sur une ligne](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1071 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      63 o ← la console, à l'allumage - la mise en route de la console
deplace_x()    158 o ← main() - une fonction fournie par le langage
main()         28 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases - attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() - trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() - efface des cases
CopierTuiles   18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()      x 2   26 o sur place, dans deplace_x() + AdresseCase (10 o)
effacer()    x 1   15 o sur place, dans deplace_x() + AdresseCase (10 o), EffacerCases (12 o)
image()      x 2    6 o sur place, dans deplace_x(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
deplace_x() x 1   19 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_x> // fait avancer une tuile sur sa ligne
#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A
```

2. UNE VARIABLE — PAS

```
uint8_t pas = 5;
```

3. UNE VARIABLE — X

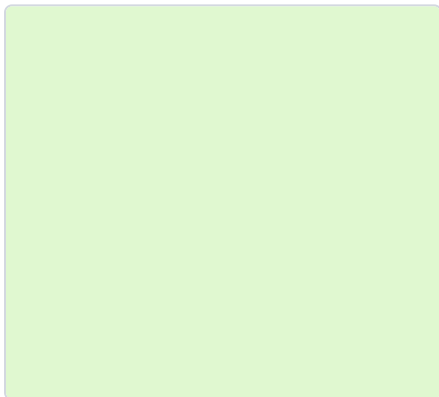
```
uint8_t x = 0;           // la colonne du B
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

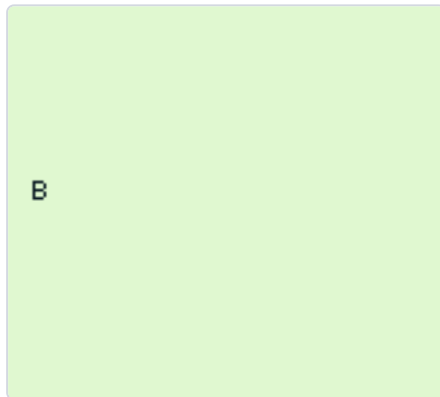
```
int main() {  
    // Les changements : la ligne (0 → 8) et la lettre (ALPHABET[1], le B).  
    x = deplace_x(x, 8, ALPHABET[1], pas);    // x vaut 5  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un B qui avance de 5 cases sur la ligne 8.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le B finit en (5, 8) (5,8)

✓ x vaut 5

Ensuite : **0.18.2. Avancer avec `deplace_x` — doublé, deux positions** — chapitre 0 : Avant tout. · `gameboy3` — écrire une cartouche Game Boy en C++